Paul Gathondu

Solo Engineer — Adaptive Learning Systems, Rust/WASM, Browser Extensions

aulgondu@gmail.com · github.com/paulgsc · pgdev.maishatu.com · github.com/paulgsc/some-ui

SUMMARY

I build and ship `some-ui`: a solo-maintained, production-shaped monorepo whose real deliverable is an adaptive, gamified tutor for Korean and low-level Rust — measured against a benchmark I can't fake, whether I actually reach TOPIK-level Korean and genuine Rust systems fluency myself. Everything else in the repo (the extension suite, the CI/CD, the release pipeline) exists to make that one product sustainable for a team of one, on a real budget, without trading engineering taste for velocity.

SOME-UI — MONOREPO, SOLE ENGINEER (2024 — PRESENT)

Situation. One developer, one budget, 61 workspace packages spanning a React/TypeScript app, a Rust/WASM game engine, and 15+ shipped browser extensions. No headcount to throw at regression risk and no budget to route every change through a frontier model — the repo has to defend its own quality bar structurally, not by brute force.

What I built and why it holds up:

- **Adaptive learning core** — designed and shipped `hangul-game-core` (Rust/WASM) and `honeycomb` (hex-grid study UI) for Korean acquisition, plus a TOPIK prep module and a typing-drill engine. Architecture changes to the curriculum model are derived in written ADRs and formal design canons **before** implementation — e.g. generalizing the engine from single-glyph to multi-token words was proved correct (an equivalence theorem, not a hope) before a line of the change landed — because the curriculum only gets harder from here and a wrong abstraction compounds.
- **CI/CD as a dependency graph, not a checklist** — Turborepo + pnpm workspaces power a PR pipeline that scopes lint/typecheck/test to changed packages and their transitive dependents (one required `ci-gate` check), backed by a separate full-repo trunk sweep and a standalone Rust CI job. Regression enforcement is real: lints, types, and Rust's clippy/cargo-deny gate merges, not a suggestion.
- **Release, for real** — Changesets drives versioned publishing with auto-generated changelogs across every workspace package; GitHub Actions handle Storybook/design-system deploys, Docker/GHCR builds for the app, and signed extension releases, all issue- and milestone-tracked through normal PR review rather than direct-to-main commits.
- **Flagship extension: `some-filter`** — a structurally isolated dark-theme and invert-filter system for arbitrary third-party pages, built around an explicit DOM-layer invariant (extension UI can never nest inside the patched page layer, by construction, not by convention) so theming a hostile page can never leak into or break the extension's own chrome. Shipped alongside 14 other extensions on shared `common`/`transport` packages.
- **Judgment over automation-as-default** — the same discipline that keeps CI honest shows up inside the product: the learning engine keeps its content model typed and explicitly boundaried (a `Stimulus`/`Answer` split, not a bag of strings) so **what data belongs where** is a design decision made once, deliberately — not something quietly decided by whatever an LLM felt like inferring from a prompt.

STACK

TypeScript · React · Rust · WebAssembly · Turborepo · pnpm · Vite · GitHub Actions · Zod · Changesets · Typst

Set in Typst from `packages/ui/resume/resume.typ` in this repository — compiled to PDF as part of its own build pipeline.